Bike Share Analysis - Comparing Toronto Neighbourhoods

Extracting Toronto Bike Station IDs by BIA and Neighbourhood

To run

This document uses interactive tools to curate station lists. To activate the map for a specific neighborhood, ensure the code chunk is set to eval: true. You can select your stations and save the IDs.

How this works

The code acts as a spatial filter for Toronto's bike data. It takes the official boundaries of a BIA (Business Improvement Area in Toronto) and its surrounding Neighbourhood, then checks the coordinates of every bike station in the city. It automatically keeps only the stations that fall inside those specific areas. This allows us to look at a clean, focused map of just the stations that matter to that local community.

What we can do with this

With these specific station lists, we can run a comparison between different parts of the city. It's a way to see if one area has more bike support and engagement than another, which helps us make a data-backed case for better infrastructure in the Beach BIA.

Official Toronto Resources

Use these links to find specific Toronto BIAs and Neighbourhoods.

BIA Names: City of Toronto BIA List & Maps

Example: "The Beach", "Broadview Danforth"

Neighbourhood Names: City of Toronto Neighbourhood Profiles

Example: "The Beaches", "Playter Estates-Danforth"

Station Locations: Bike Share Toronto System Map

To manually check for where exactly a bike share dock is location.

Static Map of Selected Stations

Using the interactive tools below, you can determine a curated list of bike stations for anywhere in Toronto. Once you have manually selected your stations, list your selected station IDs, like how it's done in the beach_picks vector below, to tell R exactly which markers to focus on for this specific area.

The `plot_static_network()` function then cross-references these IDs with the full dataset and overlays them onto official BIA and neighborhood boundaries. This produces a clear, static map that highlights your chosen network in green, providing a professional snapshot of where your picks sit within the city's infrastructure.

```
source("BikeStation_SpatialFunctions.R")
# Define your picks
beach_picks <- c("8190", "7314", "7695", "7313", "7427", "7428",
                 "7309", "7365", "7692", "7315", "7318", "7317", "7316", "7364")

beach_bia <- get_geo_data("bia", "The Beach")
beach_nbhd <- get_geo_data("neighborhood", "The Beaches")
all_stations <- get_bike_stations()

beach_categorized_data <- categorize_stations(
    stations = all_stations,
    bia_sf = beach_bia,
    nbhd_sf = beach_nbhd,
    buffer_dist = 500
  )
```

```
# Plot it
plot_static_network(
  categorized_data = beach_categorized_data,
  bia_sf = beach_bia,
  nbhd_sf = beach_nbhd,
  curated_ids = beach_picks,
  area_label = "The Beach BIA"
)
```

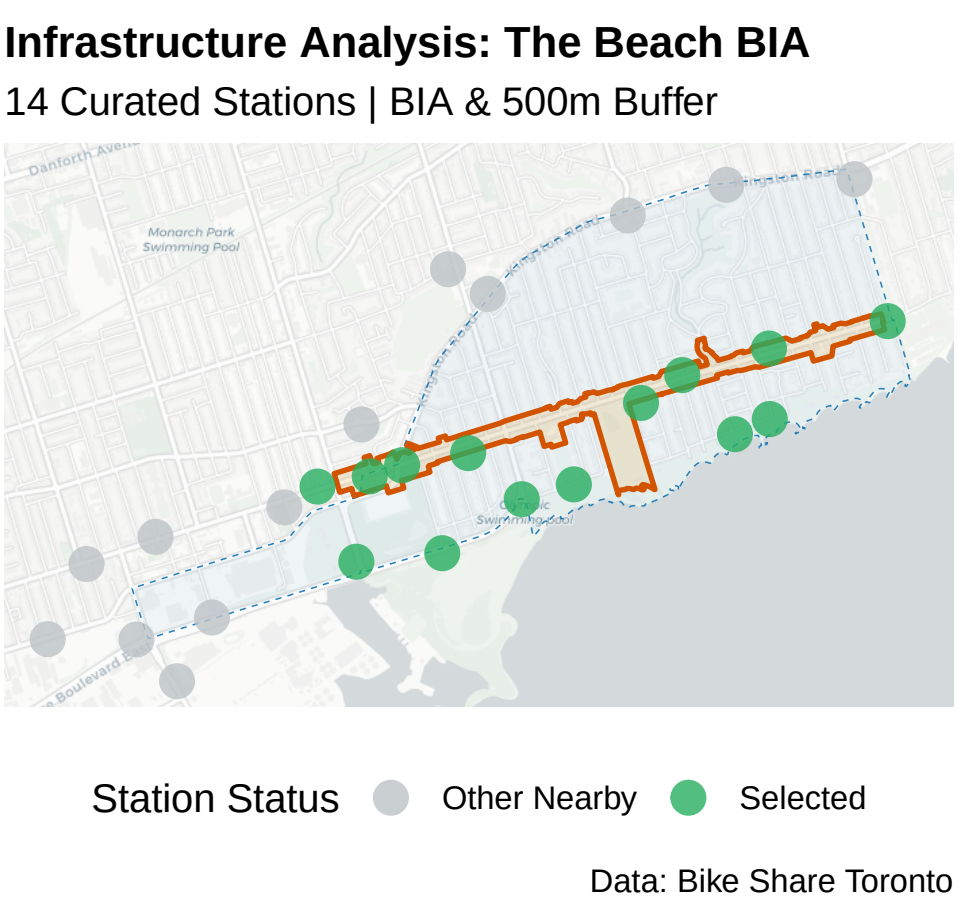

Figure 1: Curated Bike Share Infrastructure in the Beach BIA

1. The Beach & The Beaches

```
# Load custom toolkit
source("BikeStation_SpatialFunctions.R")

beach_bia <- get_geo_data("bia", "The Beach")
beach_nbhd <- get_geo_data("neighborhood", "The Beaches")
all_stations <- get_bike_stations()

beach_categorized_data <- categorize_stations(
    stations = all_stations,
    bia_sf = beach_bia,
    nbhd_sf = beach_nbhd,
```

```r
    buffer_dist = 500
  )

beach_final_ids <- collect_stations_visually(beach_categorized_data, beach_nbhd, beach_bia)
```

```r
# This prints the IDs with quotes in one line, and the count below it
if (length(beach_final_ids) > 0) {
  # We use paste0 to wrap each ID in quotes before collapsing them
  cat("Station IDs:", paste0('"', beach_final_ids, '"', collapse = ", "), "\n")
  cat("Total Count:", length(beach_final_ids))
}
```

Station IDs: "8190", "7314", "7695", "7313", "7427", "7428", "7309", "7365", "7692", "7315", "7318", "7317", "7316", "7364" Total Count: 14

2. Broadview Danforth

```r
# Fetch boundaries
broadview_bia  <- get_geo_data("bia", "Broadview Danforth")
broadview_nbhd <- get_geo_data("neighborhood", "Playter Estates-Danforth")

# Categorize and Filter (to avoid plotting all of Toronto)
broadview_categorized <- categorize_stations(
  stations = all_stations,
  bia_sf = broadview_bia,
  nbhd_sf = broadview_nbhd,
  buffer_dist = 500
)

# Run Selector Gadget
broadview_final_ids <- collect_stations_visually(
  broadview_categorized, broadview_nbhd, broadview_bia
)
```

```r
# This prints the IDs with quotes in one line, and the count below it
if (length(broadview_final_ids) > 0) {
  # We use paste0 to wrap each ID in quotes before collapsing them
  cat("Station IDs:", paste0('"', broadview_final_ids, '"', collapse = ", "), "\n")
  cat("Total Count:", length(broadview_final_ids))
}
```

Station IDs: “7351”, “7388”, “7922”, “7095”, “7094” Total Count: 5

3. Trinity-Bellwoods

```
# Fetch boundaries
trinity_bia  <- get_geo_data("bia", "Trinity-Bellwoods")
trinity_nbhd <- get_geo_data("neighborhood", "Trinity-Bellwoods")

# Categorize and Filter
trinity_categorized <- categorize_stations(
  stations = all_stations,
  bia_sf = trinity_bia,
  nbhd_sf = trinity_nbhd,
  buffer_dist = 500
)

# Run Selector Gadget
trinity_final_ids <- collect_stations_visually(
  trinity_categorized, trinity_nbhd, trinity_bia
)
```

```
# This prints the IDs with quotes in one line, and the count below it
if (length(trinity_final_ids) > 0) {
  # We use paste0 to wrap each ID in quotes before collapsing them
  cat("Station IDs:", paste0('"', trinity_final_ids, '"', collapse = ", "), "\n")
  cat("Total Count:", length(trinity_final_ids))
}
```

Station IDs: “7206”, “7541”, “7536”, “7037” Total Count: 4

4. Bloor West Village & High Park

```
# Fetch boundaries
highpark_bia  <- get_geo_data("bia", "Bloor West Village")
highpark_nbhd <- get_geo_data("neighborhood", "High Park-Swansea")

# Categorize and Filter
```

```r
highpark_categorized <- categorize_stations(
  stations = all_stations,
  bia_sf = highpark_bia,
  nbhd_sf = highpark_nbhd,
  buffer_dist = 500
)

# Run Selector Gadget
highpark_final_ids <- collect_stations_visually(
  highpark_categorized, highpark_nbhd, highpark_bia
)
```

```r
# This prints the IDs with quotes in one line, and the count below it
if (length(highpark_final_ids) > 0) {
  # We use paste0 to wrap each ID in quotes before collapsing them
  cat("Station IDs:", paste0('"', highpark_final_ids, '"', collapse = ", "), "\n")
  cat("Total Count:", length(highpark_final_ids))
}
```

Station IDs: "7460", "7086", "7438", "7224", "7221", "7654", "7461", "7084", "7741", "7688", "7223" Total Count: 11

Shiny App

This Shiny app is an interactive Geospatial Selection Tool designed to curate a custom list of Toronto Bike Share stations. It dynamically filters live station data based on official BIA (Business Improvement Area) and Neighbourhood boundaries.

Optional to use, can be used to download a list of bike station ids.

```r
library(shiny)
library(leaflet)
library(sf)
library(dplyr)
library(DT)

# 1. Load your custom toolkit
source("BikeStation_SpatialFunctions.R")

# 2. Pre-fetch names for the dropdown menus
bia_list <- opendatatoronto::get_resource("4d9216bd-71e7-4416-8d17-c2699c6354f0") %>%
```

```
  pull(AREA_NAME) %>% sort()

nbhd_list <- opendatatoronto::get_resource("0719053b-28b7-48ea-b863-068823a93aaa") %>%
  pull(AREA_NAME) %>% sort()

# --- UI SECTION ---
ui <- fluidPage(
  titlePanel("Toronto Bike Share: Professional Selector Tool"),

  sidebarLayout(
    sidebarPanel(
      p("Select a BIA and Neighbourhood. Click dots to add/remove stations from your list.")

      selectInput("selected_bia", "1. Search BIA:", choices = bia_list, selected = "The Beac
      selectInput("selected_nbhd", "2. Search Neighbourhood:", choices = nbhd_list, selected

      actionButton("recenter", "Recenter Map", icon = icon("crosshairs")),

      hr(),
      h4("Selected Stations"),
      DTOutput("selected_table"),

      br(),
      downloadButton("download_csv", "Download Selected IDs (CSV)"),
      style = "height: 90vh; overflow-y: auto;"
    ),

    mainPanel(
      leafletOutput("map", height = "900px")
    )
  )
)

# --- SERVER SECTION ---
server <- function(input, output, session) {

  selected_ids <- reactiveVal(character())
  all_stations <- reactive({ get_bike_stations() })

  # Reactive boundaries based on user search
  bia_geo  <- reactive({ req(input$selected_bia); get_geo_data("bia", input$selected_bia) })
  nbhd_geo <- reactive({ req(input$selected_nbhd); get_geo_data("neighborhood", input$selecte
```

```r
# Categorize stations for the current search area
current_view_data <- reactive({
  req(bia_geo(), nbhd_geo())
  categorize_stations(all_stations(), bia_geo(), nbhd_geo(), 500)
})

# Initialize map
output$map <- renderLeaflet({
  leaflet() %>%
    addProviderTiles("CartoDB.Positron")
})

# TRIGGER 1: Update boundaries and jump to location ONLY when search changes
observeEvent(list(input$selected_bia, input$selected_nbhd), {
  req(bia_geo(), nbhd_geo(), current_view_data())

  bbox <- st_bbox(nbhd_geo())

  proxy <- leafletProxy("map") %>%
    clearShapes() %>%
    clearMarkers() %>%
    # Static jump (no flying)
    fitBounds(
      lng1 = as.numeric(bbox[["xmin"]]), lat1 = as.numeric(bbox[["ymin"]]),
      lng2 = as.numeric(bbox[["xmax"]]), lat2 = as.numeric(bbox[["ymax"]])
    )

  proxy %>%
    addPolygons(data = nbhd_geo(), color = "blue", weight = 1, dashArray = "5, 5",
                fillColor = "blue", fillOpacity = 0.05, group = "Boundaries") %>%
    addPolygons(data = bia_geo(), color = "darkorange", weight = 2,
                fillColor = "orange", fillOpacity = 0.2, group = "Boundaries") %>%
    addCircleMarkers(data = current_view_data(),
                     layerId = ~station_id,
                     radius = 9, color = "white", fillColor = "#2c3e50",
                     fillOpacity = 0.7, weight = 1, label = ~name)

  # Redraw Green Highlights if stations were already selected
  if(length(selected_ids()) > 0) {
    sel_points <- all_stations() %>% filter(station_id %in% selected_ids())
    proxy %>% addCircleMarkers(data = sel_points, layerId = ~paste0("sel_", station_id),
                               radius = 11, color = "black", fillColor = "#00ff00",
```

```
                                weight = 2, fillOpacity = 1, group = "highlights")
  }
}, ignoreInit = FALSE)

# TRIGGER 2: Selection Logic (Does NOT move the map)
observeEvent(input$map_marker_click, {
  id <- input$map_marker_click$id
  clean_id <- gsub("^sel_", "", id)

  current_list <- selected_ids()
  new_list <- if (clean_id %in% current_list) setdiff(current_list, clean_id) else c(curre
  selected_ids(new_list)

  proxy <- leafletProxy("map") %>% clearGroup("highlights")
  if(length(new_list) > 0) {
    sel_points <- all_stations() %>% filter(station_id %in% new_list)
    proxy %>% addCircleMarkers(data = sel_points, layerId = ~paste0("sel_", station_id),
                               radius = 11, color = "black", fillColor = "#00ff00",
                               weight = 2, fillOpacity = 1, group = "highlights",
                               label = ~paste("Selected:", name))
  }
})

# TRIGGER 3: Manual Recenter
observeEvent(input$recenter, {
  req(nbhd_geo())
  bbox <- st_bbox(nbhd_geo())
  leafletProxy("map") %>%
    fitBounds(
      lng1 = as.numeric(bbox[["xmin"]]), lat1 = as.numeric(bbox[["ymin"]]),
      lng2 = as.numeric(bbox[["xmax"]]), lat2 = as.numeric(bbox[["ymax"]])
    )
})

# Table and Download Handlers
output$selected_table <- renderDT({
  all_stations() %>%
    st_drop_geometry() %>%
    filter(station_id %in% selected_ids()) %>%
    select(ID = station_id, Name = name) %>%
    datatable(options = list(dom = 'tp', pageLength = 5), rownames = FALSE)
})
```

```r
  output$download_csv <- downloadHandler(
    filename = function() { paste0("selected_bike_stations_", Sys.Date(), ".csv") },
    content = function(file) {
      write.csv(data.frame(station_id = selected_ids()), file, row.names = FALSE)
    }
  )
}

shinyApp(ui, server)
```